

CSA3023/CSE3023/CSM3023 WEB-BASED APPLICATION DEVELOPMENT

Lab Modules



WAN NURAL JAWAHIR HJ WAN YUSSOF, RABIEI MAMAT,
MOHD ARIZAL SHAMSIL MAT RIFIN
& MUHAMMAD SYAFFIQ HAZARI

Contents

List of Figures	v
1 DEVELOPMENT ENVIRONMENT SETUP FOR JAVA WEB APPLICATIONS	1
1.1 Objectives	1
1.2 Introduction	1
1.3 Software Requirements	2
1.4 Task 1: Install Java Development Kit (JDK)	2
1.4.1 Step 1: Download JDK	2
1.4.2 Step 2: Install JDK	2
1.4.3 Step 3: Verify JDK Installation	2
1.5 Task 2: Install XAMPP	3
1.5.1 Step 1: Download XAMPP	3
1.5.2 Step 2: Install XAMPP without Tomcat	4
1.5.3 Step 3: Verify XAMPP Installation	9
1.6 Task 3: Change the Default Root Password of MySQL	11
1.6.1 Step 1: Start Apache and MySQL Services	11
1.6.2 Step 2: Run Command	12
1.7 Task 4: Tomcat 9 Installation	18
1.7.1 Step 1: Download Apache Tomcat 9	18
1.7.2 Step 2: Extract Apache Tomcat 9	19
1.7.3 Step 3: Start Tomcat Server	19
1.7.4 Step 4: Verify Installation	20
1.8 Task 5: NetBeans IDE Installation	21
1.8.1 Step 1: Download Apache NetBeans 29	21
1.8.2 Step 2: Extract and Install NetBeans 29	21
1.8.3 Step 3: Configure NetBeans 29	23
1.8.4 Step 5: Configure Tomcat in NetBeans	23
1.9 Task 6: Writing a Simple Java Servlet	26
1.9.1 Step 1: Create New Project	26
1.9.2 Step 2: Create New Servlet	29
1.9.3 Step 3: Run Servlet	32
1.10 Task 7: Writing a Simple JSP Program	34
1.10.1 Step 1: Create a New Project	34
1.10.2 Step 2: Create a New JSP File	36
1.10.3 Step 3: Run JSP File	39
1.11 Task 8: Use Java Reference Datatype/Class Wrapper in JSP	41
1.11.1 Step 1: Create a New JSP File	41

1.12	Exercise	43
2	SERVLET: DATA SHARING AND DATABASE MANAGEMENT	45
2.1	Objectives	45
2.2	Introduction	46
2.3	Software Requirements	46
2.4	Task 1: Data Sharing in Servlet	46
2.4.1	Step 1: Project Setup	46
2.4.2	Step 2: Create HTML Login Page	48
2.4.3	Step 3: Create a Login Servlet	48
2.4.4	Step 4: Create an Account Servlet	51
2.4.5	Step 5: Run the Project	52
2.5	Task 2: Creating A Table in MySQL Database	53
2.5.1	Step 1: Start Apache and MySQL	53
2.5.2	Step 2: Create a New Database	53
2.5.3	Step 3: Create a New MySQL Table	53
2.6	Task 3: Setting the Environment of Web Application for Database Connection	54
2.6.1	Step 1: Create a New Web Application Project	54
2.6.2	Step 2: Prepare the MySQL Connector	55
2.7	Task 4: Using Servlets for Database CRUD Operations	57
2.7.1	Step 1: Create HTML Form for Data Entry	57
2.7.2	Step 2: Create the Insert Servlet (Create)	58
2.7.3	Step 3: Create the View Servlet (Read)	59
2.7.4	Step 4: Create the Delete Servlet (Delete)	62
2.7.5	Step 5: Create the Update Servlet (Update)	63
2.8	Task 5: Using Servlets, DAO and Java Beans for Database CRUD Operations	65
2.8.1	Step 1: Copy and Rename the Project	66
2.8.2	Step 2: Create the User JavaBean (Model)	66
2.8.3	Step 3: Create the Data Access Object (UserDAO)	68
2.8.4	Step 4: Modify the Existing Servlets (Controller)	72
2.9	Exercise	77
3	JSP: SCRIPTLET, PAGE DIRECTIVE & INCLUDE DIRECTIVE	79
3.1	Objectives	79
3.2	Introduction	79
3.3	Software Requirements	80
3.4	Task 1: Passing Data from Main JSP's Page to Other JSP's Page	80
3.4.1	Step 1: Create A New JSP Page	80
3.4.2	Step 2: Write an HTML Markup	81
3.5	Task 2: Using Mathematics Operations in JSP	84
3.5.1	Step 1: Create A New JSP Page	85
3.5.2	Step 2: Write an HTML Markup	85
3.5.3	Step 3: Run the <code>Calculator.jsp</code>	86
3.6	Task 3: Populate an Array Values into HTML's Table	89
3.6.1	Step 1: Create A New JSP Page	89
3.6.2	Step 2: Write An HTML Markup	89
3.6.3	Step 3: Run the <code>populateArray.jsp</code> File	90

3.7	Task 4: Perform Calculation of Car Loan	90
3.7.1	Step 1: Create A New HTML File	90
3.7.2	Step 2: Create An HTML Markup	91
3.7.3	Step 3: Create <code>processCalculateCarLoan.jsp</code>	92
3.8	Task 5: Using JSP Page Directive to Call Java API	93
3.9	Task 6: Use JSP Include directive for JSP Page	98
4	JSP: SCRIPTLET, EXPRESSION & STANDARD ACTIONS	103
4.1	Objectives	103
4.2	Introduction	103
4.3	Software Requirements	104
4.4	Task 1: Using JSP Scripting	110
4.4.1	Step 1: Create <code>customer.html</code>	110
4.4.2	Step 2: Create <code>processCustomer.jsp</code>	111
4.4.3	Step 3: Compile and Run the Application	113
4.5	Task 2: Using JSP (Scripting, Declaration and Expression)	115
4.5.1	Step 1: Create <code>currencyConversion.html</code>	115
4.5.2	Step 2: Create <code>processCurrency.jsp</code>	116
4.5.3	Step 3: Compile and Run the Application	118
4.6	Task 3: Using JSP Standard Action (Include and Param)	120
4.6.1	Step 1: Create <code>jspParameter.jsp</code>	120
4.6.2	Step 2: Create <code>subjectInfo.jsp</code>	122
4.6.3	Step 3: Compile and Run the Application	123
4.7	Task 4: Using JSP Standard Action (Forward)	124
4.7.1	Step 1: Create <code>forward.jsp</code>	124
4.7.2	Step 2: Create <code>forwardInfo.jsp</code>	125
4.7.3	Step 3: Compile and Run the Application	126
4.8	Task 5: Using JSP Scriptlet to Construct Business Logic	127
4.8.1	Step 1: Create Project and JSP File <code>insuranceQuotation.jsp</code>	127
4.8.2	Step 2: Design Insurance Quotation Interface	127
4.8.3	Step 3: Create <code>processInsuranceQuo.jsp</code>	128
4.8.4	Step 4: Implement Business Logic Using JSP Scriptlet	129
4.8.5	Step 5: Compile and Run the Application	130
5	JSP: JAVABEANS AND STANDARD TAG LIBRARY (JSTL)	135
5.1	Objectives	135
5.2	Introduction	136
5.3	Software Requirements	136
5.4	Task 1: Using Scriptlet to Access a Simple JavaBeans	136
5.4.1	Step 1: Create the JavaBean	136
5.4.2	Step 2: Access Bean via Scriptlet	137
5.5	Task 2: Problem Solving using JavaBeans	138
5.5.1	Step 1: Create a Calculator Bean	138
5.5.2	Step 2: Create Form and Process Page	139
5.6	Task 3: Installing JSTL Taglibs	140
5.6.1	Step 1: Download and Import JSTL Libraries	141
5.6.2	Step 2: Test the Installation and Formatting Tags	142
5.7	Task 4: Using Java Standard Tag Library (JSTL) Core Tags	144

5.7.1	Step 1: Variables and Conditionals	144
5.8	Task 5: Using JSP Standard Tag Library (JSTL) for Collections	145
5.8.1	Step 1: Create the Servlet (Controller)	145
5.8.2	Step 2: Create the View (JSTL forEach)	146
5.9	Task 6: Database Access using JSTL SQL Tags	147
5.9.1	Step 1: Create Database and Table	148
5.9.2	Step 2: Prepare the Database Driver	148
5.9.3	Step 3: Create the JSTL SQL Page	149
5.10	Exercise: Employee Payroll Display System	151
6	JSP: SAVING AND RETRIEVING DATA FROM DATABASE	153
6.1	Objectives	153
6.2	Introduction	153
6.3	Software Requirements	154
6.4	Task 1: Using JSP Page to Access a Simple MySQL Database	155
6.4.1	Step 1 - Create a table namely <code>FirstTable</code> using phpMyAdmin	155
6.4.2	Step 2 - Create <code>SampleInsertionRecord.jsp</code> to insert data in <code>FirstTable</code> table.	156
6.4.3	Step 3 - go to the database.	156
6.5	Task 2: Create Records via JSP Page	157
6.6	Task 3: Create Records Constrained by Regular Expression in JSP	159
6.6.1	Step 1 - Create a table <code>student</code> using phpMyAdmin.	159
6.6.2	Step 2 - Create <code>insertStudent.jsp</code> as a main interface to register a new student.	160
6.6.3	Step 3 - Create <code>Book</code> JavaBeans	160
6.6.4	Step 4 - Create <code>processBook.jsp</code> to insert record into the database	160
6.6.5	Step 5 - Create <code>errorBook.jsp</code> to display any error message	161
6.6.6	Step 6 - Running the program and create a new database	161
6.7	Task 4: Perform Retrieving Records Via JSP Page	161
6.8	Task 5: Create A Record Using JSP Model 1	162

List of Figures

1.1	XAMPP Download Platform	3
1.2	XAMPP Setup Wizard Startup Screen	5
1.3	XAMPP Select Components	5
1.4	XAMPP Installation Directory	6
1.5	XAMPP Language	7
1.6	XAMPP Installation Ready	7
1.7	XAMPP Installation Progress	8
1.8	XAMPP Installation Finish	8
1.9	XAMPP Control Panel	9
1.10	XAMPP Control Panel Running	10
1.11	XAMPP Dashboard	10
1.12	phpMyAdmin Interface	11
1.13	XAMPP Control Panel-Click Shell	11
1.14	Verify Password	12
1.15	Error Page	13
1.16	Apache Tomcat9 Download Page	18
1.17	Start Tomcat Server	19
1.18	Apache Tomcat Homepage	20
1.19	Apache NetBeans 29 Download Page	21
1.20	Extract netbeans-29-bin.zip	22
1.21	Netbean Executable File	22
1.22	Netbean IDE GUI	23
1.23	Configure Server	24
1.24	Add Server	24
1.25	Add Server	25
1.26	Add Server	25
1.27	Add Server	26
1.28	Create New Project	27
1.29	Create New Web Application	27
1.30	Enter Project Details	28
1.31	Create New Servlet	29
1.32	Name the Servlet	30
1.33	Configure Servlet Deployment	30
1.34	HelloServlet.java File	31
1.35	<code>processRequest()</code> Method	31
1.36	Modified <code>processRequest()</code> Method	31
1.37	Run the Servlet	32

1.38	Dialog Box Call to /HelloServlet	32
1.39	Output	33
1.40	Modified <code>doGet()</code> Method	33
1.41	Dialog Box with Passing Parameter	34
1.42	Output	34
1.43	Create New Web Application	35
1.44	Enter Project Details	36
1.45	Configure the Server	36
1.46	Create a New JSP File	37
1.47	Enter Name and Location	37
1.48	Modified JSP File	38
1.49	Compile JSP File	38
1.50	Compile JSP File	38
1.51	Run JSP File	39
1.52	Output	40
1.53	Start/Stop Apache Tomcat	40
1.54	Create <code>useJavaObject.jsp</code>	41
1.55	<code>useJavaObject.jsp</code>	42
1.56	Output	43
2.1	Create New Servlet	49
2.2	Create Login Servlet	49
2.3	Configure servlet deployment	50
2.4	Creating the “CRUDusingServlet” Project	55
2.5	Downloading MySQL Connector/J	55
2.6	Extracting the MySQL Connector/J Archive	56
2.7	Adding External Library to Project	56
2.8	Importing MySQL Connector/J Library	57
3.1	Create new JSP	81
3.2	New JSP	81
3.3	<code>memberRegister.jsp</code> Interface	82
3.4	Filling input <code>memberRegister.jsp</code>	84
3.5	Output in <code>memberProcessing.jsp</code>	84
3.6	Create new JSP	85
3.7	New JSP	85
3.8	<code>calculator.jsp</code> Interface	86
3.9	Key-in Filename	89
3.10	Output from Array	90
3.11	Creating New HTML File	91
3.12	Interface Output for Car Loan Calculation HTML	91
3.13	Input for Car Loan Calculation	92
3.14	Output for Car Loan Calculation	93
3.15	New JSP	93
3.16	Key-in Filename	94
3.17	Save All	96
3.18	Run	96
3.19	Success Message	96

3.20	ArrayList Output	97
3.21	Tomcat Before Run	97
3.22	Tomcat After Run	97
3.23	Tomcat Success Message	98
3.24	mainPage.jsp	98
3.25	headerPage.jsp Output	98
3.26	footerPage.jsp Output	99
3.27	mainPage.jsp Output	99
4.1	Creating customer.html	111
4.2	Interface of customer.html	111
4.3	Creating processCustomer.jsp	112
4.4	Example input in customer.html	114
4.5	Output after processing customer input	114
4.6	Creating currencyConversion.html	115
4.7	Interface of currencyConversion.html	116
4.8	Creating processCurrency.jsp	116
4.9	Interface of currencyConversion.html	119
4.10	Input example in currencyConversion.html	119
4.11	Processing result of currency conversion	120
4.12	Creating jspParameter.jsp	121
4.13	Creating subjectInfo.jsp	122
4.14	Output of jspParameter.jsp	123
4.15	Creating forward.jsp	124
4.16	Creating forwardInfo.jsp	125
4.17	Output of forwarded information	126
4.18	Creating insuranceQuotation.jsp	127
4.19	Insurance Quotation Interface	128
4.20	Creating processInsuranceQuo.jsp	128
4.21	Insurance Quotation Output	131
5.1	Download jstl-1.2.jar	141

Lab Module 6

JSP: SAVING AND RETRIEVING DATA FROM DATABASE

6.1 Objectives

The primary goal of this lab is to enable students to integrate JavaServer Pages with a relational database management system. Upon completion of this lab, students will be able to:

1. **Establish Database Connectivity:** Successfully connect a JSP application to a MySQL database using the JDBC (Java Database Connectivity) driver.
2. **Perform CRUD Operations:** Implement SQL transactions to Create (insert) and Read (retrieve) records through JSP scriptlets and standard actions.
3. **Implement Data Validation:** Use Regular Expressions within JavaBeans to constrain and validate user input before database insertion.
4. **Apply Design Patterns:** Utilize JSP Model 1 architecture and Data Access Objects (DAO) to manage business logic and separate database interactions from the presentation layer.
5. **Handle Errors:** Create dedicated error pages using JSP directives to manage and display exceptions during database transactions.

6.2 Introduction

In modern web development, the ability to persist and retrieve data is fundamental. This lab exercise focuses on JSP-Database integration, teaching students how to move beyond static data and simple form processing to create data-driven web applications.

Students will learn the lifecycle of a database transaction in a web environment: starting services via a control panel, creating database schemas and tables using phpMyAdmin, and writing Java code to bridge the gap between the user interface and the storage layer. The lab also introduces advanced concepts like JavaBeans for data encapsulation and the DAO pattern for cleaner, more maintainable code structures.

6.3 Software Requirements

To complete the tasks in this manual, the following software environment is required:

- **Web & Database Server Bundle: XAMPP:** to provide the Apache web server and MySQL database.
- **Database Management Tool: phpMyAdmin:** for schema and table creation
- **Integrated Development Environment (IDE):** NetBeans.
- **Database Driver:** MySQL Java Connector.
- **Web Browser:** Any modern browser (e.g., Google Chrome, Mozilla Firefox, Microsoft Edge) for testing and viewing output.

6.4 Task 1: Using JSP Page to Access a Simple MySQL Database

Objective	Write a JSP that can insert data to MYSQL database as “Welcome to access MySQL database with JSP...!” and also display steps of how to connect with MYSQL database.
Problem Description	• Create a table known as FirstTable using database schema CSA3203, create the first column as a character length 45. • Create SampleInsertionRecord.jsp page to process and acknowledge the user upon inserting record in the database
Estimated Time	20 minutes

6.4.1 Step 1 - Create a table namely **FirstTable** using phpMyAdmin

1. Start XAMPP control panel.
2. Start the Apache web server.
3. Start the MySQL database
4. Click the *Admin* button for MySQL.
5. Go to *Database*'s tab.
6. Key-in as **CSA3203** and click *Create* button.
7. Database schema successfully created.
8. Use any tool to manipulate the SQL statement. Create table **FirstTable** in the created database schema.
9. Create *FirstTable*'s table.
10. Execute the SQL statement
11. Table successfully created.

6.4.2 Step 2 - Create `SampleInsertionRecord.jsp` to insert data in *FirstTable* table.

1. Go to `C:\CSA3023` and create sub-directory as *Lab 6*
2. Go to NetBeans.
3. Go to File -> New Project
4. Select Java Web -> Web Application
5. Click the *Next* button.
6. Type Project Name: *Lab6*
7. Choose Project Location `C:\CSA3023\Lab6`.
8. Click the *Next* button.
9. Select Server: *Apache Tomcat*.
10. Select Java EE Version: Java EE 7 Web.
11. Click the *Next* button.
12. Click the *Finish* button.
13. Create a new JSP's page for and rename `SampleInsertionRecord`.
14. Type header1 as ***Lab 6 - Task 1 - Sample Insertion records into MySQL through JSP's page.***
15. To support the database driver, we need to use JSP Page Directive to provide directions and instructions to the container.
16. Write the following code:
17. Save and compile `SampleInsertionRecord.jsp` file.
18. Run the `SampleInsertionRecord.jsp` file and sample of output is shown below:

6.4.3 Step 3 – go to the database.

1. Go to Database schema (`CSA3023`)
2. Click on-> `CSA3023` -> `FirstTable` -> then Browser (see the data is already there!!)

Reflection

1. How do you want to submit specific information from one form to next form?
2. What happened if the field name you specify in `request.getParameter("field_name")` in second page is different from the field name you defined in first page?

6.5 Task 2: Create Records via JSP Page

Objective	Using JSP to insert records retrieve from MySQL database.
Problem Description	1. Create a table known as Author using database schema CSA3023 using these attributes: • authno as a character length 15 and must be primary key name. • address as a character length 40. • city as a character length 40. • state as a character length 40. • zip as a character length 40. 2. Create <code>insertAuthor.jsp</code> as a main interface to register a new author. 3. Create <code>processAuthor.jsp</code> page to process and acknowledge the user upon inserting record in the database.
Estimated Time	40 minutes

1. Use any tool to manipulate the SQL statement. Create table **author** in CSA3023 database schema.
2. Create **author**'s table.
3. Execute the SQL statement.
4. Table successfully created.
5. Create a new JSP page and rename as `insertAuthor`.

6. Write an HTML code to
 - (a) Display six (6) labels and textfields representing *Author No*, *Name*, *Address*, *City*, *State* and *Zip* (in the combo box).
 - (b) Create a *Submit* button and *Cancel* button.
 - (c) Upon submission, redirect the page to `processAuthor.jsp` page.
 7. Produce the following output:
 8. Go to *Lab6* project folder.
 9. Right click -> *New* -> *Java Class*.
 10. Click *Java Class*.
 11. Rename *Class Name* as *Author* and *Package* as *lab6.com*.
 12. Define **six (6)** instance variables for *Author* class.
 13. Define the *getter* and *setter* method for corresponding attributes.
 14. Create a new JSP page as `processAuthor`.
 15. Add the page directive to `processAuthor.jsp` page.
 16. Create an *author*'s object using JSP Standard Action tag.
 17. Assign data entry from page `insertAuthor.jsp` page into *author*'s bean.
 18. Load the database driver and create a connection to the database.
 19. Create a *PreparedStatement*'s object.
 20. Execute the query and display the result.
 21. Close database connection.
 22. Save and compile `processAuthor.jsp` file.
- IMPORTANT:** Please add **MySQL Java connector** to your project before running the program.
23. Run `insertAuthor.jsp` page.
 24. Key-in the record.
 25. Click *Submit* button.
 26. The record will save in the database, and user get a notification.

Reflection

1. What have you learnt from this exercise?
2. Define step by step before you successfully perform the transaction in a database.

6.6 Task 3: Create Records Constrained by Regular Expression in JSP

Objective	Using JSP Standard Action, scriptlets and regular expression to insert records retrieve from MySQL database.
Problem Description	1. Create a table known as Student using database schema CSA3023 using these attributes: • stuid as a character length 15 and must be primary key name. • stuname as a character length 50. • stuprogram as a character length 40. • address as a character length 40. 2. Create <code>insertStudent.jsp</code> as a main interface to register new book. 3. Create <code>processStudent.jsp</code> page to process and acknowledge the user upon inserting record in the database. 4. Create <code>displayStudent.jsp</code> page to populate records. 5. Create <code>errorStudent.jsp</code> page to handle an error.
Estimated Time	50 minutes

6.6.1 Step 1 – Create a table **student** using phpMyAdmin.

1. Create table **student** in CSA3023 database schema.
2. Execute the SQL statement.
3. Table successfully created.

6.6.2 Step 2 – Create `insertStudent.jsp` as a main interface to register a new student.

1. Create a new JSP page and rename as `insertStudent`.
2. Write an HTML code to
 - (a) Display three (3) labels and textfields representing *Author No*, *Name*, and *Program* (in the combo box).
 - (b) The first field must be started with captain letters the numbers input
(Use the following regular expression in *Book JavaBeans* file)
 - (c) Create a *Submit* button and *Cancel* button.
3. Produce the following output:

6.6.3 Step 3 – Create *Book JavaBeans*

1. Go to *Lab6* project folder.
2. Right click -> *New* -> *Java Class*.
3. Click *Java Class*.
4. Rename *Class Name* as *Book* and *Package* as *lab6.com*.
5. Define **three (3)** instance variables for *Book* class.
6. Define the *getter* and *setter* method for corresponding attributes.
7. Define *getter* and *setter* method plus regular expression for *stuno* attribute.

6.6.4 Step 4 – Create `processBook.jsp` to insert record into the database

1. Create a new JSP page as `processStudent`.
2. Add the page directive to `processAuthor.jsp` page.
3. Create a *student's* object using JSP Standard Action tag.
4. Assign data entry from page `insertStudent.jsp` page into *Student's* bean.
5. Load the database driver and create a connection to the database.
6. Create a *PreparedStatement's* object.

7. Execute the query and display the result.
8. Close database connection.

6.6.5 Step 5 – Create `errorBook.jsp` to display any error message

1. Create a new JSP page as `errorStudent`.
2. Define the page directive to declare that this is an error page.
3. Complete remaining of code.
4. Save and run `errorStudent.jsp` file.

6.6.6 Step 6 – Running the program and create a new database

1. Run `insertStudent.jsp` page.
2. Key-in the record.
3. Click `Submit` button.
4. The record will be saved in the database, and user will get a notification.

Reflection

1. What have you learnt from this exercise?
2. Define step by step before you successfully perform the transaction in a database.

6.7 Task 4: Perform Retrieving Records Via JSP Page

Objective	Use Java Scriptlet to query a list of records.
Problem Description	Retrieve student records and populate in the table.
Estimated Time	30 minutes

1. Run program *`insertStudent.jsp`* from Task 3.

2. Insert the following records;

UK12489	Ahmad Salam	BSc with IM
UK56789	Rosnah Azman	BSc Soft. Eng.
UK67342	Liew Cheng Huat	BSc in Robotics

3. Go to *Lab6* project.
4. Create a new JSP file.
5. Key-in filename as `queryStudent`.
6. Rename title as **Lab 6 - Task 4**.
7. Rename `h1` as **Lab 6 - Task 4: Retrieving record via JSP Page**.
8. Use JSP page directive to include the information such as content type, and use Java SQL API.
9. Use a Java scriptlet to create a simple structure of HTML table.
10. Then, load the database driver and connect to the database.
11. Create Statement for the query.
12. Perform query to retrieve records from the Student's table.
13. Fetch the record into HTML table.
14. Close the database connection.
15. Enhance the CSS for the table.
16. Save *queryStudent.jsp*, then run.
17. You should get the following output.

Reflection

1. What have you learnt from this exercise?
2. Explain the differences when using *Statement()* and *PreparedStatement()*.

6.8 Task 5: Create A Record Using JSP Model 1

Objective	Use JavaBeans to perform SQL transaction.
Problem Description	Create a sample web form to register the Marathon event.
Estimated Time	40 minutes

1. Choose Project *Lab6*.
2. Create a new JSP file.
3. Key-in filename as `registerMarathon`.
4. Prepare the following Graphical User Interface (GUI).
5. Create a JavaBeans `Marathon`.
6. Create a `Database` class that has two methods; `getConnection()`, and `closeConnection()`.
7. Create a `MarathonDAO` class to perform SQL transaction for business object `Marathon` and store it into package `lab6.com`.
8. Create a new file name known as `processMarathon.jsp`.
9. Import related classes in package `lab6.com`.
10. Instantiate an object `Marathon`.
11. Create a Java Scriptlet to invoke respective object for inserting record in `marathon`'s table.
12. Save and run `processMarathon.jsp`, and key-in related record.
13. The output will appear in a web browser.

Reflection

1. What have you learnt from this exercise?
2. Describe the benefits of using JavaBeans.

Exercise

Implement user login

1. Create a table known as **userprofile** using database schema CSA3023 using these attributes
 - **username** as a character length 15 and must be primary key
 - **password** as a character length 10
 - **firstname** as varchar(50)
 - **lastname** as varchar(50)
2. Create **insertUser.html** as the main interface to register a new user.
3. Create **processUser.jsp** page to process the record.
4. Create **login.jsp** page to login to the system.
5. Create **doLogin.jsp** to validate username and password. If validation succeeds, redirect the page to **main.jsp** page that displays the username, firstname, and lastname
6. However if fails, redirect the page to **login.jsp** with message 'Invalid username or password..!'